iDEFT: Intelligent Backup-Node Selection for Fault-Tolerant Network Function State Replication

Abstract—Network Function Virtualization (NFV) systems that provide fault tolerance through state replication (OpenNF, Pico Replication, FTMB, REINFORCE, and DEFT among them) all replicate a primary network function's (NF) state to a backup instance, yet none of them specify how that backup instance is chosen. In DEFT, for example, the Failure Detection Unit simply "assigns" a new secondary NF with no stated criterion. This paper closes that gap with iDEFT (intelligent DEFT), which designs and evaluates three backup-selection strategies as drop-in replacements for DEFT's static, hand-assigned mapping: a multi-criteria greedy heuristic (health, remaining capacity, network latency, hop-count), a globally optimal assignment under per-backup capacity constraints solved exactly via the Hungarian algorithm, and a Proximal Policy Optimization (PPO) reinforcement-learning policy trained on the same objective with invalid-action masking. We evaluate all three, together with naive baselines (DEFT's shipped static mapping, uniform-random, round-robin, and a worst-case concentrated assignment), by driving DEFT's real, unmodified two-phase-commit (2PC) backup implementation under controlled concurrent load, across four topology scales (9, 16, 32, and 128 hosts) with rigorous multi-seed statistical testing (Mann-Whitney U tests, rank-biserial effect sizes, bootstrap confidence intervals). We find that feature-aware methods reliably avoid degraded backup hosts and substantially reduce tail latency compared to blind load-balancing once hosts differ in quality, that a well-designed deterministic heuristic tracks the Hungarian-optimal bound closely at every scale tested, and that the learned RL policy matches this performance up to 32 hosts but requires non-default hyperparameters to avoid a degenerate policy collapse at 128 hosts; even after that fix, it still under-performs the heuristic on load-balancing precision at that scale. We report this honestly as a precisely characterized limitation rather than a solved problem: RL learns host-quality discrimination faster than it learns load-balancing precision as the action space grows. The code, trained models, and the saved output of every experiment are available for reproduction at https://bijoy111.github.io/ideft/.

Index Terms—network function virtualization, fault tolerance, state replication, backup node selection, reinforcement learning, software-defined networking

I. Introduction

Network Function Virtualization (NFV) replaces dedicated hardware middleboxes with virtualized network functions (NFs) running on commodity servers, enabling elastic scaling and fault tolerance that fixed hardware cannot provide [1]. A stateful NF's correctness depends on preserving per-flow and global state across failures and load-driven scaling events. The dominant approach to fault tolerance for stateful NFs is replication: a primary NF instance forwards its state updates to one or more backup (secondary) instances, which take over processing if the primary fails [2]–[4]. DEFT [5] extends this line of work with a distributed, controller-free two-phase-commit (2PC) protocol that achieves both elastic scaling and fault tolerance with strong consistency on global state updates, without the single point of failure a central controller introduces.

A. The Gap: Backup Selection Is Never Specified

Across this entire line of work, backup assignment is treated as solved by fiat rather than as a decision with consequences. DEFT's Failure Detection Unit "assigns NF B as the new primary and NF C as the new secondary NF" [5] with no stated criterion for choosing C. REINFORCE [4] designates "a predecessor node and a backup node" per NF without describing a selection rule. OpenNF [6], Split/Merge [7], StatelessNF [8], and S6 [9] all describe how state is migrated or shared once a target instance is known, never how that target is chosen in the first place. In DEFT's own reference implementation, this manifests concretely: the shipped static mapping for its 9-host evaluation topology concentrates 3 of 5 primaries onto a single backup host, with no load or health awareness whatsoever.

This is a consequential gap. The entire point of replication for fault tolerance is that the backup must be available and capable of taking over. If the backup itself is overloaded, unhealthy, or already serving many other primaries, the replication mechanism's guarantees are only as good as an arbitrary assignment allows. Elastic scaling compounds this: as NF instances are added and removed under varying load, a backup pool's composition changes continuously, and a static or naively-assigned mapping cannot track it. We deliberately scope this work to the fault-tolerance half of state management, namely replication, because that is where the backup-selection decision lives. Choosing a target for state migration during elastic scaling is a related but distinct decision, and our assignment formulation could extend to it, but we do not evaluate it here.

B. Contributions

We present iDEFT (intelligent DEFT), addressing backup-node selection directly and using DEFT as the concrete system under study (though the problem and our methods generalize to any primary/backup replication scheme). Specifically, we:

1) Formulate backup selection as a feasibility- and load-aware assignment problem, and design three solutions spanning a deterministic multi-criteria greedy heuristic, an exact globally-optimal assignment (Hungarian algorithm) under capacity constraints, and a learned PPO policy with invalid-action masking during training.
2) Build a reproducible evaluation harness that drives DEFT's real, unmodified 2PC backup implementation under controlled concurrent load via XML-RPC, isolating the backup-selection mechanism from confounding Mininet [10]/SDN-controller environment issues (§IV).
3) Evaluate all methods against DEFT's shipped static mapping and three naive baselines (random, round-robin, worst-case concentrated) across four topology scales (9 to 128 hosts), with rigorous multi-seed statistical testing rather than single point-estimate comparisons.
4) Measure not only tail latency but also RPC throughput, failure rate, and per-mapping selection-time overhead, to directly support the elastic-scaling claim that selection itself must be fast.
5) Report an honest, mechanistically-characterized limitation of the RL approach at 128-host scale, together with the two escalating hyperparameter fixes attempted, rather than suppressing a negative result.

The remainder of this paper is organized as follows. §II surveys elastic and fault-tolerant NF state management and RL-based NFV resource management, and states our positioning precisely. §III describes the backup-selection methods. §IV describes the experimental testbed and statistical methodology. §V presents results. §VI discusses threats to validity. §VII concludes.

II. Related Work

A. Elastic State Management

State management during NF scaling events follows two strategies: state migration and migration avoidance.

State migration. Split/Merge [7] introduces partitioned (per-flow) and coherent (global) NF state; coherent updates are periodically merged and so remain eventually consistent [11]–[13] across replicas, though in-flight traffic arriving at the source NF during migration is dropped. OpenNF [6] uses move, copy, and share operations to achieve varying consistency levels in NF state sharing, with share achieving strong [14] or strict [15], [16] consistency; both systems require a central controller to coordinate migration.

Migration avoidance. StatelessNF [8] and S6 [9] avoid migration by decoupling state from processing to varying degrees. StatelessNF [17] makes states globally accessible via a remote data store, adding per-packet latency and introducing a single point of failure. S6 stores states in a distributed shared-object space addressed via a Chord-like key/object layer [18], minimizing (but not eliminating) migration through method-call shipping, local caching, and object replication with eventual consistency.

B. Fault-Tolerant State Management

Pico Replication [2] uses frequent checkpointing [19] for loss-free failure recovery, at the cost of not handling in-flight packets that arrive during checkpointing. FTMB [3] instead uses infrequent checkpointing with packet-level logging and replay, favoring high steady-state throughput over checkpoint frequency. REINFORCE [4] combines both strategies: packets are buffered at a predecessor node and replayed to a backup node on failure, with lazy checkpointing (transmitting only a timestamp table per batch) except when a batch contains non-deterministic packets [20], which force strict checkpointing. All three systems assume a backup already exists for each primary and describe only the replication/recovery protocol; none describe how that backup is chosen. DEFT [5] unifies elastic scaling and fault tolerance in one system via a distributed 2PC protocol with no central controller dependency (unlike [6], [7], [21]), but inherits the same gap: backup assignment is a bookkeeping step in its Failure Detection Unit, not a studied decision.

C. Reinforcement Learning for NFV Resource Management

A separate line of work applies reinforcement learning to NFV resource-management decisions, almost entirely focused on primary placement, scaling, or embedding rather than backup/replica selection. Cand-RL [22] uses PPO to optimize availability-aware Service Function Chain (SFC) placement. DDQN-VNFPA [23] uses Double Deep Q-Networks for VNF placement in SDN/NFV-enabled networks, reducing rejection rates. He et al. [24] jointly optimize VNF placement and routing with an attention-augmented DRL agent. MAPLE [25] proactively adjusts VNF placement via clustering and integer linear programming. AREL3P [26] uses end-to-end performance prediction with RL for autonomous placement in 5G networks. DeepOpt [27] combines DRL with graph neural networks for topology-generalizing VNF placement, and Lun et al. [28] address large state spaces and slow convergence in SFC placement optimization. Kibalya et al. [29] apply RL to the placement of stateful VNFs specifically, but the state under consideration is the primary's own execution state, not a fault-tolerance replica relationship between two NF instances. The nearest neighbor to our work is Avgeris et al. [30], an RL-based self-healing agent that repositions VNFs after a failure or resource-contention event has already been detected: a reactive decision, made after the fact, rather than the proactive backup-assignment decision this paper studies as part of setting up replication itself. Two recent surveys, Lim et al. [31] on RL for virtual network embedding and Saha et al. [32] on RL for network slice scaling and placement, both

catalogue placement, scaling, and embedding as the RL-addressed sub-problems of NFV resource management; backup/replica selection for fault tolerance appears in neither, underscoring that it is an open sub-problem rather than a solved one being re-attempted here.

D. Positioning of This Work

Outside NFV, the principle that replica placement itself deserves algorithmic attention is well established in distributed storage systems: CRUSH [33] computes failure-domain- and load-aware placement of replicated data without a centralized lookup table, and Dynamo [34] uses consistent hashing with preference lists to spread replicas while accounting for heterogeneous node capacity. Both treat "which node holds the copy" as a first-class, criteria-driven decision: precisely the decision DEFT and its predecessors leave unaddressed for backup NF assignment. This paper closes that gap for stateful NFV fault tolerance, building directly on DEFT's 2PC replication mechanism, and evaluates the closure with a level of statistical rigor (multi-seed testing, effect sizes, bootstrap confidence intervals) not present in DEFT's own original evaluation or in the RL-for-NFV placement literature surveyed above, which typically reports point-estimate comparisons against a single baseline run.

III. Backup-Selection Methods

Let P be the set of primary NFs and H the set of all hosts (a superset of P; any host not currently a primary for a given flow, and not an infrastructure host such as the client or stamper, is a feasible backup for that primary, including hosts that are primaries for other flows). For each primary $p \in P$ we must choose a backup $b \in H \setminus \{p\}$. Each candidate host h has a dynamic resource state: health $\text{health}(h) \in [0, 1]$, remaining capacity $\text{cap}(h)$ out of $\text{cap}_{\max}(h)$, CPU/storage availability, and a topology-derived hop-count and latency to p.

A. Baselines

Static (DEFT's shipped mapping). The hand-assigned mapping DEFT ships with for its 9-host reference topology, regenerable on demand for direct comparison.

Concentrated. Every primary assigned to the same single feasible backup: a scale-independent, maximally naive worst case, generalizing DEFT's own static mapping's tendency to over-concentrate load onto one host.

Random. A uniformly random feasible backup per primary, seeded for reproducibility: a sanity floor any non-random method should beat.

Round-robin. Cycles through the sorted list of all feasible hosts, skipping a candidate only when it equals the primary itself. Load-balances blindly, with zero awareness of host health, capacity, or latency.

B. Heuristic: Multi-Criteria Greedy Scoring

For each primary (processed in a fixed order), the heuristic scores every feasible candidate c as

$$\begin{aligned}\text{score}(p, c) = w_h \cdot \text{health}(c) + w_k \cdot \frac{\text{cap}(c)}{\text{cap}_{\max}(c)} \\ - w_\ell \cdot \hat{\ell}(p, c) - w_d \cdot \hat{d}(p, c)\end{aligned} \tag{1}$$

where $\hat{\ell}$ and $\hat{d}$ are the min-max-normalized latency and hop-count from p to c among p's candidates, and default weights $(w_h, w_k, w_\ell, w_d) = (0.35, 0.35, 0.20, 0.10)$ favor health and capacity, which are what actually differentiate candidates once a topology has heterogeneous backup quality. The greedy assignment picks $\arg\max_c \text{score}(p, c)$ for each primary in turn, then applies a load penalty to the chosen host (cap $-= 20$; health $*= 0.95$) before moving to the next primary. This penalty is the mechanism that spreads load across the backup pool without any learning component.

C. Optimal: Capacity-Constrained Hungarian Assignment

To obtain an upper bound the heuristic and RL policy can be judged against, we solve the assignment problem exactly. Each backup h is duplicated into $k = \lceil |P|/|H \setminus P| \rceil$ capacity-constrained columns; the cost matrix entry for primary p and column j (backup h) is $-\text{score}(p, h)$, or a large penalty if h is infeasible for p. We solve this exactly via the Hungarian algorithm [35] (scipy.optimize.linear_sum_assignment [36]), yielding a globally optimal assignment under the capacity constraint, something no greedy or online method can guarantee in general.

D. RL: PPO with Invalid-Action Masking

We train a Proximal Policy Optimization [37] agent (stable-baselines3 [38] over a gymnasium [39] environment) on the same scoring objective as the heuristic. One episode assigns a backup to every primary in sequence, under one random resource realization; the observation is each host's $[\text{health}, \text{cap}/\text{cap}_{\max}, \text{cpu}, \text{storage}]$ concatenated with a one-hot encoding of the currently-active primary, and the action space is $\text{Discrete}(|H|)$. The reward is $\text{score}(p, c)$ for a feasible choice, an overload penalty proportional to how many primaries already share that backup, and a large fixed penalty (-10) for an infeasible choice, so invalid actions are discouraged during training, not merely corrected at inference time as in a naive post-hoc-masking design. A fresh random resource realization is drawn every episode so the learned policy generalizes across resource realizations rather than memorizing one instance. At inference, the deterministic policy is rolled out once per primary, falling back to the best-scoring feasible candidate in the rare case the policy still selects an infeasible action.

IV. Experimental Methodology

A. Testbed: RPC-Isolation, Not Physical Mininet

We measure backup-selection quality by driving the real, unmodified Backup and Primary 2PC classes from DEFT's implementation directly over XML-RPC, with each backup host running as its own OS process on localhost. This is a deliberate departure from a physical Mininet network path, which we attempted first: the development environment's kernel lacked the openvswitch kernel module and the htb traffic-control queueing discipline, no SDN controller was included in the original repository, and, decisively, TCLink bandwidth throttling did not engage on the available kernel, so there was no real resource constraint for contention to manifest against over the physical network path. The RPC-isolation testbed, by directly exercising the actual backup RPC server under controlled concurrent load, gives a more direct and reproducible measurement of the contention mechanism in question, at the cost of not validating full end-to-end network behavior (switching, real link bandwidth, packet loss); see §VI.

Each primary fires a burst of simultaneously in-flight RPC calls rather than one sequential call-then-wait stream, matching how concurrent primaries would load a shared backup in production. Three implementation pitfalls were identified and corrected during harness development: (1) running backup servers as threads within one process shares a GIL, confounding the very contention effect being measured, so servers now run as separate OS processes; (2) creating a fresh RPC connection per call forces a new TCP handshake every time, dominating measurement with connection-thrash noise rather than queueing/RPC latency, so each burst worker keeps one persistent connection for all its calls, with retry-with-jittered-backoff on transient connection resets; and (3) an early version built a single host-resource context per seed and passed that same object to every method, but the heuristic and RL selectors apply an in-place load-decay penalty to each host they pick, so any method evaluated after them scored against an already-depleted context and its measured mapping depended on execution order, so each method now receives its own copy of the context.

B. Scenarios

Homogeneous (9 hosts). DEFT's real reference topology, 5 primaries, synthetic resource values drawn independently per random seed with no engineered heterogeneity.

Scaled heterogeneous (16, 32, 128 hosts). Synthetic star topologies at increasing scale (10/20/80 primaries; 6/12/48 backup-eligible hosts), with $\lfloor |H \setminus P|/3 \rfloor$ of the backup-eligible hosts deliberately degraded (health = 0.05, capacity ratio $\approx$ 0.03, plus an artificial +50 ms per-call server-side delay simulating a genuinely overloaded machine). This isolates whether feature-aware methods avoid known-bad hosts, independent of load-balancing alone, and whether the finding holds as the candidate pool grows by roughly an order of magnitude.

TABLE I
Homogeneous scale (9 hosts, 5 primaries), n = 15,000 calls/method.

Method	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
baseline	57.33	10.67	41.24	1071.77
concentrated	94.99	6.81	221.69	3125.54
random	45.98	12.35	47.50	1063.38
round_robin	35.67	18.46	54.27	1035.42
heuristic	41.06	17.48	50.68	1054.36
optimal	37.00	19.63	56.03	1034.23
rl	36.20	18.88	54.47	1038.38

C. Statistical Methodology

We draw multiple independent topology/resource realizations (context seeds) rather than repeating trials of one fixed instance, since repeated trials of a single instance only capture RPC-timing noise, not whether a result generalizes across problem instances; reported numbers aggregate across all (context seed × trial) combinations. Per-trial method execution order is randomized so no method is systematically favored or disadvantaged by background system load at a particular point in a long run. We use the two-sided Mann-Whitney U test for pairwise significance and rank-biserial correlation as an accompanying effect size, since with sample sizes in the tens of thousands, p-values are almost always significant regardless of practical magnitude ($|r| \approx 0.1/0.3/0.5$ conventionally denote small/medium/large effects). We report bootstrap 95% confidence intervals on the mean (5000 resamples).

V. Results

A. Homogeneous Scale

Table I reports per-call latency across 5 context seeds × 3 trials (200 calls/primary, burst=10). Consistent with the preamble above, we read these comparisons by effect size rather than by p: round_robin vs. optimal ($r = +0.031$) and heuristic vs. rl ($r = +0.036$) are both negligible, and round_robin vs. rl does not even reach significance ($r = +0.008$); round_robin vs. heuristic is small ($r = -0.029$). baseline/concentrated vs. everything else is small-to-medium ($r \in [0.08, 0.40]$). At this small, homogeneous scale, any method that avoids gross load concentration performs similarly well: the feature-aware methods show no meaningful advantage over blind round-robin, since (as Table III confirms) contention itself, not feature-awareness, drives the difference here; the real gap is between naive-concentration policies and everything else.

B. Scaled Heterogeneous: 16 Hosts

Table II isolates feature awareness specifically: 2 of 6 backup-eligible hosts are deliberately degraded. concentrated, heuristic, optimal, and rl never route a primary

TABLE II
Scaled heterogeneous, 16 hosts (10 primaries), $n = 30{,}000$ calls/method.

Method	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
concentrated	157.37	4.89	1029.37	3361.56
random	124.97	16.77	360.19	1069.84
round_robin	106.94	22.41	360.26	1040.29
heuristic	39.11	19.55	53.68	1050.06
optimal	37.25	21.46	54.44	1036.45
rl	37.71	21.47	56.97	1039.31

TABLE III
Latency vs. backup load, pooled across all methods/seeds.

Scenario	Load	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
Homog.	1	35.35	17.94	52.07	1036.52
	2	53.44	10.23	49.57	1074.48
	3	74.77	8.14	221.96	1090.20
	4	111.85	6.12	1018.58	3140.93
16-host	1	58.19	20.67	355.99	1038.28
	2	49.88	13.35	49.22	1071.96
	3	238.40	14.08	361.21	1390.65
	9	172.04	4.59	1036.05	3374.23

TABLE IV
Selection overhead (ms) per full mapping computation.

Method	9 hosts	16 hosts	32 hosts	128 hosts
concentrated	0.005	0.013	0.055	0.731
random	0.021	0.029	0.084	0.639
round_robin	0.004	0.005	0.015	0.030
heuristic	0.055	0.161	0.646	7.260
optimal	0.097	0.208	0.913	8.943
rl	1.81	3.76	11.29	49.64

to a degraded host across any of 5 seeds; round_robin hits a degraded host on 2 of 10 assignments in every seed (it cycles blindly with no way to know which hosts are bad); random is inconsistent (0 to 4 of 10, depending on seed). round_robin vs. heuristic is small-to-medium ($r = -0.1465$); heuristic vs. optimal is negligible-to-small ($r = +0.0586$, heuristic marginally behind optimal); heuristic vs. rl is negligible ($r = +0.0673$); optimal vs. rl does not reach significance ($r = +0.0089$). concentrated vs. everything is large ($r \in [0.47, 0.56]$). This is the core empirical result: feature-aware backup selection provides a real, mechanistically explained, statistically robust improvement over blind load-balancing once hosts differ in quality, the realistic case for elastic scaling in production, where backup capacity is added and removed under varying load and health conditions, not a static uniform pool.

C. Mechanism Check: Latency vs. Backup Load

Table III pools latency by backup_load (the number of primaries sharing a given backup) across all methods and seeds, isolating whether sharing itself, independent of which method produced the sharing pattern, drives latency. Mean latency rises with load in every scenario we ran, and by load 4 in the homogeneous scenario p99 has roughly tripled relative to load 1, consistent with TCP accept-queue backlog overflow at Python's SimpleXMLRPCServer (default backlog, single-threaded) once enough concurrent connections pile onto one backup; this is a genuine architectural property of DEFT's unmodified backup implementation, not a benchmark artifact. The mean is the stable signal here: the load-3 p99 in particular varies by nearly 3× across runs of this identical configuration, since it is a tail statistic over the smallest pool in the table. In the 16-host scenario, load 9 (concentrated piling 9 of its 10 primaries onto one backup, the tenth being that backup's own primary) shows sharply degraded mean and p99 relative to load 1–2, reproducing the same mechanism at a more extreme sharing ratio.

D. Selection Overhead

"Elastic scaling" requires the selection algorithm itself to be fast: a method that matches the optimal bound on placement quality but takes seconds to compute is a materially worse deployment story than one that is comparably fast. Table IV reports the mean wall-clock time to compute one full mapping, across all four topology sizes (30 repeats each, first call excluded as warmup), and Figure 1 plots the same data on log–log axes. Even the slowest method computes a full 128-host mapping in about 50 ms (rl, 49.6 ms mean), and every other method in under 10 ms, negligible relative to any realistic failure-detection interval (typically hundreds of milliseconds to seconds in NF failure-recovery literature [40], [41]). rl is consistently slowest (one PPO forward pass per primary, plus framework call overhead); its jump at 128 hosts (49.6 ms, versus 11.3 ms at 32 hosts) is the direct cost of the wider $[256, 256]$ network needed to fix the policy collapse described in §V-F, on top of the growth in primary count. heuristic and optimal grow roughly linearly/superlinearly with host count (the Hungarian algorithm is $O(n^3)$) but remain below 9 ms at 128 hosts. None of the feature-aware methods introduce a selection-latency cost that would offset their placement-quality advantage in practice.

E. Scalability Trend: 32 Hosts

Figure 2 plots mean and p99 latency across the full four-point scale sweep; the two subsections that follow walk through the 32- and 128-host points it summarizes. Table V reports the 32-host point (3 context seeds × 2 trials, 150 calls/primary, burst=8, reduced from the 16-host protocol to keep the larger run tractable on a single machine; see §VI). The 16-host finding holds at 2× scale: concentrated, heuristic, optimal, and rl again never route to a degraded host in any seed; round_robin hits one in 4 of 20 assignments, every seed. round_robin vs. heuristic remains small ($r = -0.1422$), similar in direction and magnitude to the 16-host result; the three feature-aware methods remain statistically close to each other

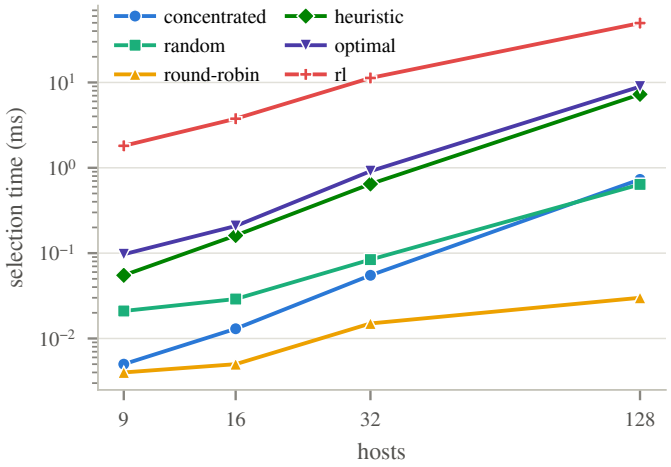


Fig. 1. Selection overhead per full mapping computation (log–log axes; data of Table IV). Even the slowest method (rl, 49.6 ms at 128 hosts) remains well below realistic failure-detection intervals.

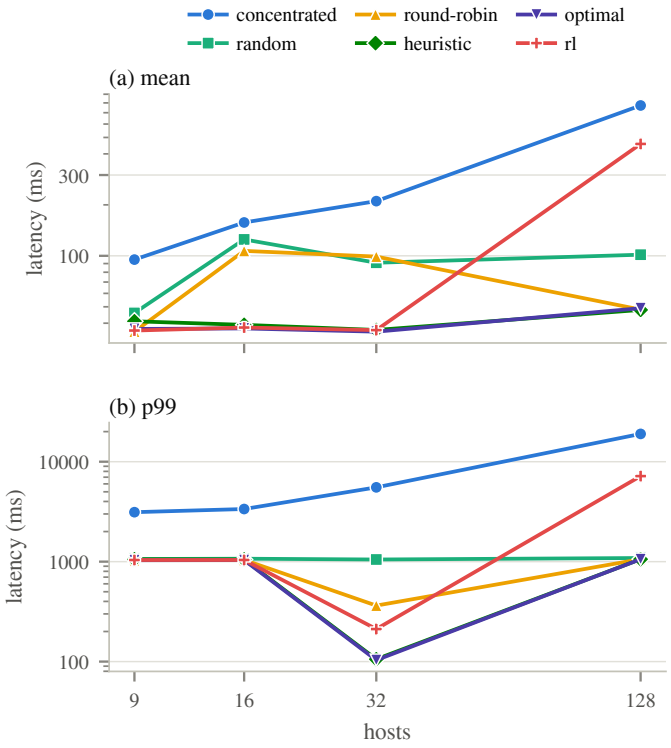


Fig. 2. Per-call latency vs. topology scale: (a) mean, (b) p99 (log axes). The 9-host point is the homogeneous scenario (its baseline method, which exists only there, is omitted); 16/32/128 hosts are the scaled heterogeneous scenarios of Tables II, V, and VI. heuristic and optimal stay flat across the sweep; concentrated degrades monotonically; rl tracks the feature-aware methods until 128 hosts.

($r \in [-0.07, 0.04]$ pairwise); concentrated vs. everything is large and stronger than at 16 hosts ($r \in [0.71, 0.76]$, vs. $[0.47, 0.56]$), since piling 19 of the 20 primaries onto one backup pushes the single-threaded backup server further into queue backlog. Notably, round_robin's deterministic

TABLE V
Scaled heterogeneous, 32 hosts (20 primaries), $n = 18{,}000$ calls/method.

Method	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
concentrated	210.35	4.28	1053.62	5538.62
random	91.00	24.49	358.65	1050.90
round_robin	98.97	28.78	359.29	363.41
heuristic	36.49	26.49	58.41	105.63
optimal	35.65	27.70	58.59	103.48
rl	36.41	25.86	54.58	211.59

TABLE VI
Scaled heterogeneous, 128 hosts (80 primaries), $n = 240{,}000$ calls/method.

Method	Mean (ms)	Median (ms)	p95 (ms)	p99 (ms)
concentrated	772.41	4.05	3325.13	18971.94
random	101.54	32.20	357.33	1087.47
round_robin	48.13	30.95	69.15	1058.35
heuristic	47.92	30.97	67.57	1058.17
optimal	49.06	31.38	71.57	1057.86
rl	457.86	26.65	232.01	7178.12

mapping is not resource-seed dependent, so its degraded-host hit rate is not a random draw; it is a fixed consequence of how Python's alphabetical (not numeric) sort of host names happens to align with wherever the degraded hosts sit in that ordering. §V-F shows this exact coincidence flips at 128 hosts, which is important context for interpreting round_robin's apparent performance at any single scale.

F. Scalability Trend Continued: 128 Hosts

Table VI reports the 128-host point at the original 16-host trial count (5 context seeds × 3 trials, 200 calls/primary, burst=10) rather than a reduced protocol, to keep this the most statistically comparable large-scale point. This run required a benchmark-harness hardening: the original 3-attempt retry budget assumed a worst case of roughly 10 primaries sharing one backup; at 79 primaries sharing one backup (concentrated), Python's default SimpleXMLRPCServer accept backlog (5) was overwhelmed badly enough that calls exhausted the retry budget and crashed the experiment outright. We hardened this to 8 attempts with jittered exponential backoff, recording a call that still fails after all retries as a failure (a 10 s sentinel latency) rather than crashing, the same robustness a real production 2PC client would need. This changes the harness's failure handling, not the backup server under test.

Concentrated sits at the edge of outright failure at this scale. In the reported run, a burst of calls against the single backup serving 79 other primaries (concentrated, context seed 2) exhausted the 8-retry budget and were recorded as failures at the 10 s sentinel latency; the other four seeds of the same configuration completed every call. Either way, concentrated's mean latency remains more than an order of magnitude above every other method, with p99 at roughly 19 s of genuine queueing delay. That whether

calls fail at all swings across otherwise-identical seeds is part of the finding: at this contention level, concentrated's behavior is not merely bad but unstable. This is a concrete finding about DEFT's shipped backup implementation: using Python's default accept backlog, it cannot reliably serve roughly 79 concurrent primaries sharing one backup even with client-side retry-with-backoff, a genuine scalability ceiling of the implementation, not a benchmark artifact.

Round-robin, heuristic, and optimal are practically indistinguishable at this scale. round_robin's mapping happens to hit zero degraded-host assignments at 128 hosts (the alphabetical-sort coincidence noted in §V-E), so it is not penalized here for the mechanism this scenario is designed to test. Their means sit within about 2% of one another (47.9, 48.1, and 49.1 ms for heuristic, round_robin, and optimal), and since these three methods' mappings are fully determined by topology and context seed, that residual spread is RPC-timing noise rather than a quality difference. This is not a reversal of our core claim but its clearest illustration: round-robin's outcome is uncontrolled, scoring badly at 16/32 hosts and well at 128 hosts purely by naming coincidence, whereas heuristic and optimal avoid degraded hosts by construction regardless of scale, naming, or noise.

RL: an attempted fix, with partial success. The original rl_model_scaled128 (400,000 timesteps, stable-baselines3's default entropy coefficient of 0 and a $[64, 64]$ network) converged to a near-degenerate policy: across the five context seeds it used only 1–5 distinct backups out of 80, funnelling almost every primary onto the same single host (w101) and triggering the same accept-backlog failure as concentrated. Since PPO's entropy bonus is zero by default, nothing discourages a policy from locking onto one action early once it looks decent on average, a plausible root cause, compounded by a $[64, 64]$ network's limited capacity to represent a policy conditioned on which of 80 one-hot-encoded primaries is active. We made two escalating retraining attempts: an intermediate one (entropy coefficient 0.01, network $[128, 128]$, 600k timesteps) that improved the spread only partially, and the fix that produced the reported model (entropy coefficient 0.02, network $[256, 256]$, larger rollout and batch sizes, 1M timesteps, the rl row of Table VI). Figure 3 contrasts the degenerate baseline with this final model, the two configurations shipped as reproducible checkpoints.

The final fix repairs one of the policy's two failings completely and the other only in part. On host quality it is total: zero degraded-host assignments across all 5 seeds (0 of 400), matching heuristic/optimal exactly and generalizing across every seed tested. On load balancing it improves markedly but stops short of the feature-aware baselines. The policy spreads the 80 primaries across 24–30 distinct backups depending on seed, against the heuristic's 74–76, and in the tighter seeds it still funnels as many as

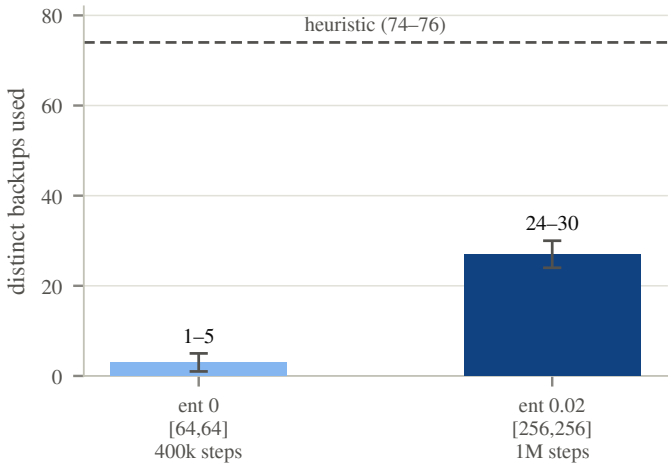


Fig. 3. Distinct backups used by the PPO policy across the 80 primaries at 128 hosts, for the degenerate baseline and the final fixed model of §V-F (the two configurations shipped as reproducible checkpoints). Bar height marks the midpoint of the per-seed range (whiskers span the range across the 5 context seeds). The entropy/capacity fix eliminates degraded-host assignments entirely and lifts the spread from 1–5 to 24–30 distinct backups, but still leaves it short of the heuristic, which places the same 80 primaries on 74–76 distinct backups at a maximum load of 2.

28 of the 80 onto one favoured host (peak per-backup load ranges 9–28 across seeds), versus heuristic/optimal's near-uniform 1–2 per backup (most of the heuristic's backups carry exactly one primary). This concentration shows up only in the tail: rl's median latency (26.6 ms) is in fact the lowest of any method in Table VI, but its hot host drives mean 457.86 ms and p99 7178.12 ms, roughly 7–10× worse than heuristic/optimal/round-robin, though still far better than the original collapse, which crashed the harness entirely.

That split is this paper's sharpest finding about RL at scale, and it is a mixed one. The same policy that learns host-quality discrimination perfectly learns load-balancing precision only partially: it spreads far better than the collapsed baseline, yet still leans on a handful of favoured hosts. The mechanism is visible in the reward: its health and capacity terms label which hosts are bad, and the network learns that completely, but nothing penalizes concentration directly, and the implicit health/capacity decay never fully substitutes for it, so a few labelled-good hosts can absorb a disproportionate share of the flow set without the policy paying enough for it. We report this as a precise, mechanistic characterization of an unresolved limitation rather than a vague "RL doesn't scale" claim: the two properties are learned at radically different rates as the action space grows. Reward shaping that directly penalizes load concentration is the natural next step and was not attempted here.

G. Throughput and Failure Rate

Tables VII and VIII complement per-call latency with the other two metrics DEFT's original evaluation re-

TABLE VII
Throughput/failure rate, homogeneous scale (5 seeds × 3 trials).

Method	Mean throughput (calls/s)	Mean failure rate (%)
baseline	1176.0	0.38
concentrated	1158.1	0.70
random	1180.5	0.26
round_robin	1172.9	0.06
heuristic	1197.3	0.14
optimal	1159.2	0.07
rl	1175.1	0.09

TABLE VIII
Throughput/failure rate, 16-host scaled scenario (5 seeds × 3 trials).

Method	Mean throughput (calls/s)	Mean failure rate (%)
concentrated	1191.4	2.57
random	1297.6	0.62
round_robin	1288.0	0.25
heuristic	1310.0	0.29
optimal	1358.8	0.16
rl	1352.0	0.17

ports: throughput and packet drop. We measure each primary's burst workers for a fixed 3-second wall-clock window (rather than a fixed call count) and report successful calls/second and the percentage of calls that time out or fail even after retry. This testbed is RPC-level (TCP), not DEFT's own packet-level input-buffer-overflow mechanism, which requires the Mininet/stamped-packet path this testbed deliberately bypasses; "failure rate" here is evidence of the same underlying phenomenon (an overloaded backup failing to keep up with demand), not a literal reproduction of DEFT's packet-drop metric.

At homogeneous scale we draw no conclusion from throughput: the measurement is too noisy at this sample size to support one in either direction. Only 3 of the 21 pairwise comparisons in the reported run reached $p < 0.05$ (all three involving heuristic, at $p \in [0.03, 0.05]$ with small effect sizes), and none corresponds to a stable ordering. Each method contributes only 15 aggregate measurements here (5 seeds × 3 trials), against the tens of thousands of individual calls behind each latency test, so the test has little power to resolve the few-percent differences in Table VII, and the verdict is correspondingly unstable: aggregate throughput at this scale is dominated by background system load rather than by the mapping under test. We report Table VII for completeness and treat the 16-host scenario below, where contention is severe enough to produce a large and stable effect, as the only informative throughput result. The one consistent signal even here is failure rate: concentrated's 0.70% is the highest of any method, while the feature-aware methods stay at 0.07–0.14%.

At 16 hosts, contention is severe enough that one part of the throughput picture becomes the most robust result in this section. concentrated vs. everything is large and significant ($r \in [0.63, 0.78]$, all $p < 0.004$), and its failure rate (2.57%) is clearly elevated over every other method's (0.16–0.62%). Gross load concentration is therefore visible from the throughput side, not only from tail latency.

The degraded-host effect is a subtler matter, and the honest characterization is that aggregate throughput resolves it only partially. In the reported run optimal and rl each significantly out-throughput round_robin ($r = 0.63$ and 0.65, $p < 0.004$), consistent with round_robin pairing 2 of its 10 primaries with a degraded backup in every seed; yet heuristic vs. round_robin does not reach significance ($p = 0.38$), despite heuristic avoiding degraded hosts exactly as optimal and rl do. The mechanism explains the fragility: only 2 of round_robin's 10 primaries are actually paired with a degraded backup, and their reduced individual throughput is averaged into the other 8 primaries' normal throughput at the aggregate level, leaving a signal small enough that run-to-run noise decides the verdict. Tail latency is dominated by exactly the slow calls that throughput's mean-based aggregation smooths over, and resolves the same effect consistently at every scale we tested. This is why we treat latency as the primary metric throughout and throughput as corroborating rather than replacing evidence: throughput reliably resolves gross contention, but only marginally resolves the finer degraded-host effect that this paper is actually about.

VI. Threats to Validity

RPC-isolation testbed, not the physical Mininet network. As described in §IV, we exercise DEFT's real backup RPC server directly rather than over a physical/emulated network path, after the available Mininet environment could not provide a genuine bandwidth constraint for contention to manifest against. This gives a direct, reproducible measurement of the contention mechanism at the cost of not validating full end-to-end network behavior (switching, real link bandwidth, packet loss).

Single physical machine. All "hosts" are OS processes on one machine; OS-level scheduling noise is a real, visible component of tail latency. Multi-seed aggregation and large sample sizes mitigate but do not eliminate this.

Synthetic resource values. Health, capacity, CPU, and storage availability are drawn from configured ranges, not measured from real workload telemetry.

RL requires scale-dependent hyperparameters. A PPO policy's observation/action space is fixed to a given host count; a differently-sized topology needs a freshly trained model, and as shown in §V-F, the hyperparameters that suffice at 16 and 32 hosts produce a degenerate policy at 128 hosts. We identify the precise remaining limitation as load-balancing precision, not host-quality discrimination, but a reward function directly penalizing load concentration was not attempted. The reported RL numbers are moreover tied to the specific trained checkpoints we release: retraining even at the same seed under a different stable-baselines3/PyTorch version, or at a different host

count, yields a different policy, so the RL rows describe the behavior of these released models rather than a version-independent guarantee of the method.

Round-robin's degraded-host exposure is coincidental, not stable. round_robin's cycling interacts with Python's alphabetical host-name sort in a way that happens to hit the degraded range at 16/32 hosts but misses it entirely at 128 hosts. Conclusions about round-robin's relative latency should be read as "uncontrolled/coincidental exposure to bad hosts," not a stable "∼20% worse" rate; the consistent, load-bearing claim is that heuristic/optimal/RL avoid degraded hosts by construction in every scenario tested, a guarantee round-robin cannot offer in any of them.

Retry-policy inconsistency across scale points. The 128-host run required a harness hardening (3 to 8 retry attempts) after the original budget crashed under 80-primary contention. The 9/16/32-host results predate this fix but never exhausted the original budget in practice, so they should be unaffected, but were not re-run under the identical harness version.

Scale sweep is three points, not a continuous curve, with the 32-host point run at a reduced trial count (3 seeds $\times$ 2 trials $\times$ 150 calls vs. $5 \times 3 \times 200$ at 16 and 128 hosts) to keep it tractable. The qualitative findings hold at all three points, but a continuous sweep with fully matched trial counts would be needed for a rigorous asymptotic scalability claim.

Multi-seed generalization covers resources, not topology structure. Our context-seed sweep varies synthetic resource draws independently per seed, but the underlying star-topology graph shape is deterministic for a given host/primary count; a genuinely different graph shape (multiple hubs, uneven fan-out) is untested.

VII. Conclusion and Future Work

Every major NF state-replication system we surveyed (OpenNF, Split/Merge, StatelessNF, S6, Pico Replication, FTMB, REINFORCE, and DEFT) treats backup-node assignment as solved by fiat rather than as a decision with measurable consequences. We have closed this gap concretely with iDEFT, designing and rigorously evaluating three backup-selection strategies (a multi-criteria heuristic, an exact Hungarian-optimal bound, and a PPO policy) against DEFT's own static mapping and three naive baselines, by driving DEFT's real, unmodified 2PC implementation under controlled contention across four topology scales with statistically rigorous, multi-seed testing. Feature-aware methods reliably avoid degraded backup hosts and substantially reduce tail latency once hosts differ in quality, a well-designed deterministic heuristic tracks the exact optimal bound closely at every scale we tested, and reinforcement learning matches this performance up to 32 hosts but requires careful, scale-dependent hyperparameter tuning to avoid a policy collapse at 128 hosts; even after that fix, it still trails the heuristic on load-balancing precision specifically, not on host-quality discrimination. We report this as a precisely characterized, currently open limitation rather than a solved problem or a silently-buried failure.

Future work includes: reward shaping that directly penalizes load concentration for the RL policy at large scale; validating these findings over a physical or emulated network path with genuine bandwidth constraints; extending the multi-seed evaluation to vary topology structure, not just resource values; and a continuous scalability sweep with matched trial counts across an uninterrupted range of host counts.

Acknowledgment

This work builds directly on the DEFT system [5] and its publicly available implementation.

References

[1] B. Han, V. Gopalakrishnan, L. Ji, and S. Lee, "Network function virtualization: Challenges and opportunities for innovations," IEEE communications magazine, vol. 53, no. 2, pp. 90–97, 2015.

[2] S. Rajagopalan, D. Williams, and H. Jamjoom, "Pico replication: A high availability framework for middleboxes," in Proceedings of the 4th annual Symposium on Cloud Computing, 2013, pp. 1–15.

[3] J. Sherry, P. X. Gao, S. Basu, A. Panda, A. Krishnamurthy, C. Maciocco, M. Manesh, J. Martins, S. Ratnasamy, L. Rizzo et al., "Rollback-recovery for middleboxes," in Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, 2015, pp. 227–240.

[4] S. G. Kulkarni, G. Liu, K. Ramakrishnan, M. Arumaithurai, T. Wood, and X. Fu, "Reinforce: Achieving efficient failure resiliency for network function virtualization based services," in Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies, 2018, pp. 41–53.

[5] M. M. Shahriyar, G. Saha, B. Bhattacharjee, and R. Reaz, "Deft: distributed, elastic, and fault-tolerant state management of network functions," in 2023 19th International Conference on Network and Service Management (CNSM). IEEE, 2023, pp. 1–7.

[6] A. Gember-Jacobson, R. Viswanathan, C. Prakash, R. Grandl, J. Khalid, S. Das, and A. Akella, "Opennf: Enabling innovation in network function control," ACM SIGCOMM Computer Communication Review, vol. 44, no. 4, pp. 163–174, 2014.

[7] S. Rajagopalan, D. Williams, H. Jamjoom, and A. Warfield, "{Split/Merge}: System support for elastic execution in virtual middleboxes," in 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13), 2013, pp. 227–240.

[8] M. Kablan, A. Alsudais, E. Keller, and F. Le, "Stateless network functions: Breaking the tight coupling of state and processing," in 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17), 2017, pp. 97–112.

[9] S. Woo, J. Sherry, S. Han, S. Moon, S. Ratnasamy, and S. Shenker, "Elastic scaling of stateful network functions," in 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18), 2018, pp. 299–312.

[10] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks, 2010, pp. 1–6.

[11] W. Vogels, "Eventually consistent: Building reliable distributed systems at a worldwide scale demands trade-offs? between consistency and availability." Queue, vol. 6, no. 6, pp. 14–19, 2008.

[12] S. Burckhardt, "Principles of eventual consistency," 2014.

[13] P. Bailis and A. Ghodsi, "Eventual consistency today: Limitations, extensions, and beyond," Communications of the ACM, vol. 56, no. 5, pp. 55–63, 2013.

[14] M. Vukolic, "Eventually returning to strong consistency." IEEE Data Eng. Bull., vol. 39, no. 1, pp. 39–44, 2016.

[15] S. V. Adve and K. Gharachorloo, "Shared memory consistency models: A tutorial," computer, vol. 29, no. 12, pp. 66–76, 1996.

[16] D. Mosberger, "Memory consistency models," ACM SIGOPS Operating Systems Review, vol. 27, no. 1, pp. 18–26, 1993.

[17] M. Szalay, M. Nagy, D. Géhberger, Z. Kiss, P. Mátray, F. Németh, G. Pongrácz, G. Rétvári, and L. Toka, "Industrial-scale stateless network functions," in 2019 IEEE 12th International Conference on Cloud Computing (CLOUD). IEEE, 2019, pp. 383–390.

[18] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, "Chord: a scalable peer-to-peer lookup protocol for internet applications," IEEE/ACM Transactions on networking, vol. 11, no. 1, pp. 17–32, 2003.

[19] J. S. Plank, M. Beck, G. Kingsley, and K. Li, Libckpt: Transparent checkpointing under unix. Computer Science Department, 1994.

[20] C. Cachin, S. Schubert, and M. Vukolić, "Non-determinism in byzantine fault-tolerant replication," arXiv preprint arXiv:1603.07351, 2016.

[21] A. Dixit, F. Hao, S. Mukherjee, T. Lakshman, and R. Kompella, "Towards an elastic distributed sdn controller," ACM SIGCOMM computer communication review, vol. 43, no. 4, pp. 7–12, 2013.

[22] G. L. Santos, P. T. Endo, T. Lynn, D. Sadok, and J. Kelner, "A reinforcement learning-based approach for availability-aware service function chain placement in large-scale networks," Future Generation Computer Systems, vol. 136, pp. 93–109, 2022.

[23] J. Pei, P. Hong, M. Pan, J. Liu, and J. Zhou, "Optimal vnf placement via deep reinforcement learning in sdn/nfv-enabled networks," IEEE Journal on Selected Areas in Communications, vol. 38, no. 2, pp. 263–278, 2019.

[24] N. He, S. Yang, F. Li, S. Trajanovski, L. Zhu, Y. Wang, and X. Fu, "Leveraging deep reinforcement learning with attention mechanism for virtual network function placement and routing," IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 4, pp. 1186–1201, 2023.

[25] O. A. Wahab, N. Kara, C. Edstrom, and Y. Lemieux, "Maple: A machine learning approach for efficient placement and adjustment of virtual network functions," Journal of Network and Computer Applications, vol. 142, pp. 37–50, 2019.

[26] M. Bunyakitanon, X. Vasilakos, R. Nejabati, and D. Simeonidou, "End-to-end performance-based autonomous vnf placement with adopted reinforcement learning," IEEE Transactions on Cognitive Communications and Networking, vol. 6, no. 2, pp. 534–547, 2020.

[27] P. Sun, J. Lan, J. Li, Z. Guo, and Y. Hu, "Combining deep reinforcement learning with graph neural networks for optimal vnf placement," IEEE Communications Letters, vol. 25, no. 1, pp. 176–180, 2020.

[28] T. Lun, H. Lanqin, L. Qinyi, and T. Qi, "Virtual network function placement optimization algorithm based on improve deep reinforcement learning," Journal of Electronics & Information Technology, vol. 43, no. 6, pp. 1724–1732, 2021.

[29] G. Kibalya, J. Serrat, J.-L. Gorricho, D. G. Bujjingo, J. Sserugunda, and P. Zhang, "A reinforcement learning approach for placement of stateful virtualized network functions," in 2021 IFIP/IEEE International Symposium on Integrated Network Management (IM). IEEE, 2021, pp. 672–676.

[30] M. Avgeris, A. Leivadeas, and I. Lambadaris, "A reinforcement-learning self-healing approach for virtual network function placement," in NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium. IEEE, 2023, pp. 1–5.

[31] H.-K. Lim, I. Ullah, Y.-H. Han, and S.-Y. Kim, "Reinforcement learning-based virtual network embedding: A comprehensive survey," ICT Express, vol. 9, no. 5, pp. 983–994, 2023.

[32] N. Saha, M. Zangooei, M. Golkarifard, and R. Boutaba, "Deep reinforcement learning approaches to network slice scaling and placement: A survey," IEEE Communications Magazine, vol. 61, no. 2, pp. 82–87, 2023.

[33] S. A. Weil, S. A. Brandt, E. L. Miller, and C. Maltzahn, "Crush: Controlled, scalable, decentralized placement of replicated data," in Proceedings of the 2006 ACM/IEEE conference on Supercomputing, 2006, pp. 31–es.

[34] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles, 2007, pp. 205–220.

[35] H. W. Kuhn, "The hungarian method for the assignment problem," Naval research logistics quarterly, vol. 2, no. 1-2, pp. 83–97, 1955.

[36] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright et al., "Scipy 1.0: fundamental algorithms for scientific computing in python," Nature methods, vol. 17, no. 3, pp. 261–272, 2020.

[37] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.

[38] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," Journal of Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021.

[39] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG et al., "Gymnasium: A standard interface for reinforcement learning environments," arXiv preprint arXiv:2407.17032, 2024.

[40] P. Gill, N. Jain, and N. Nagappan, "Understanding network failures in data centers: measurement, analysis, and implications," in Proceedings of the ACM SIGCOMM 2011 Conference, 2011, pp. 350–361.

[41] R. Potharaju and N. Jain, "Demystifying the dark side of the middle: a field study of middlebox failures in datacenters," in Proceedings of the 2013 conference on Internet measurement conference, 2013, pp. 9–22.